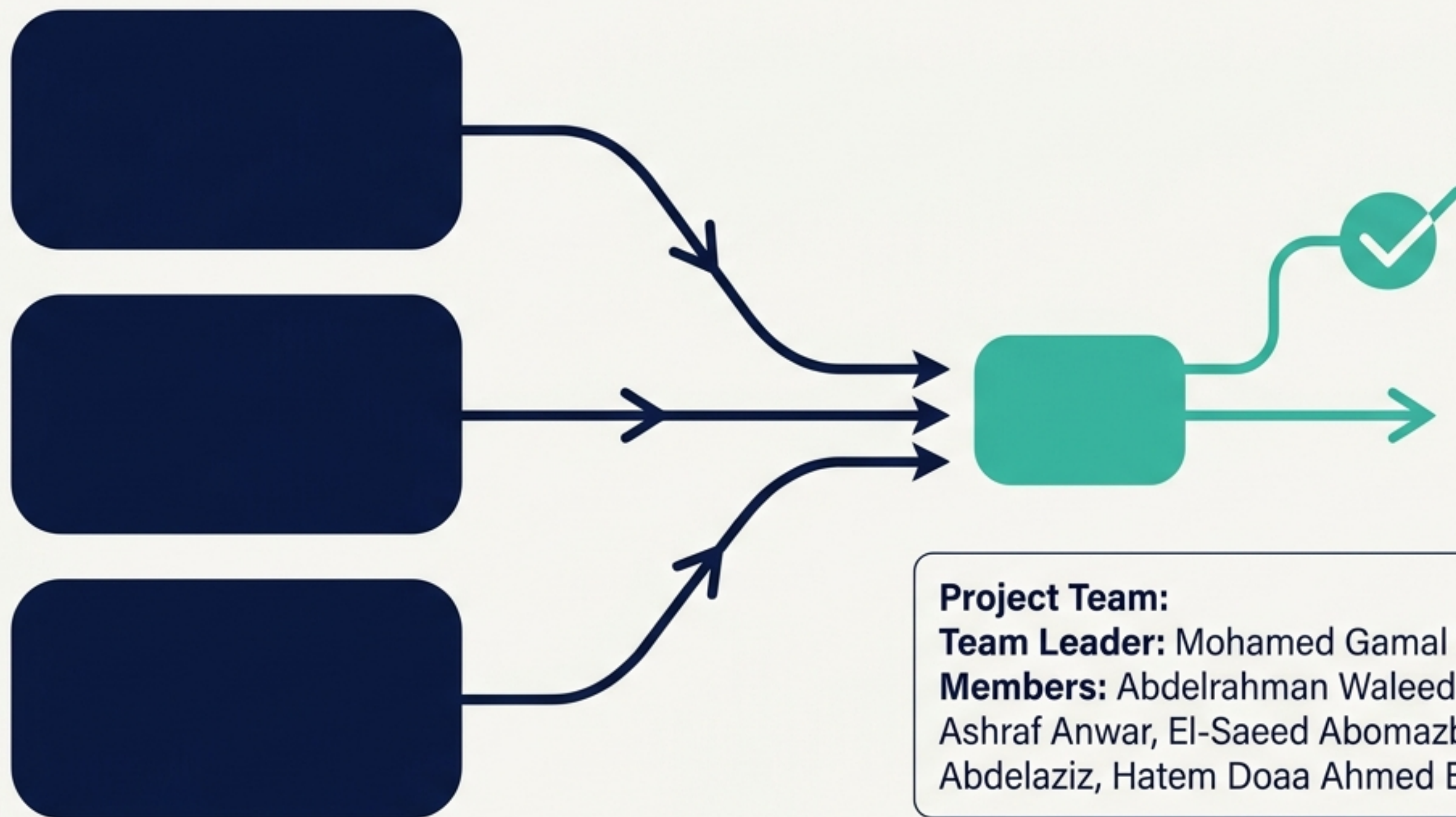


Shrinking Data, Expanding Access

Building a Huffman Compression Web App using Python and Flask



Project Team:

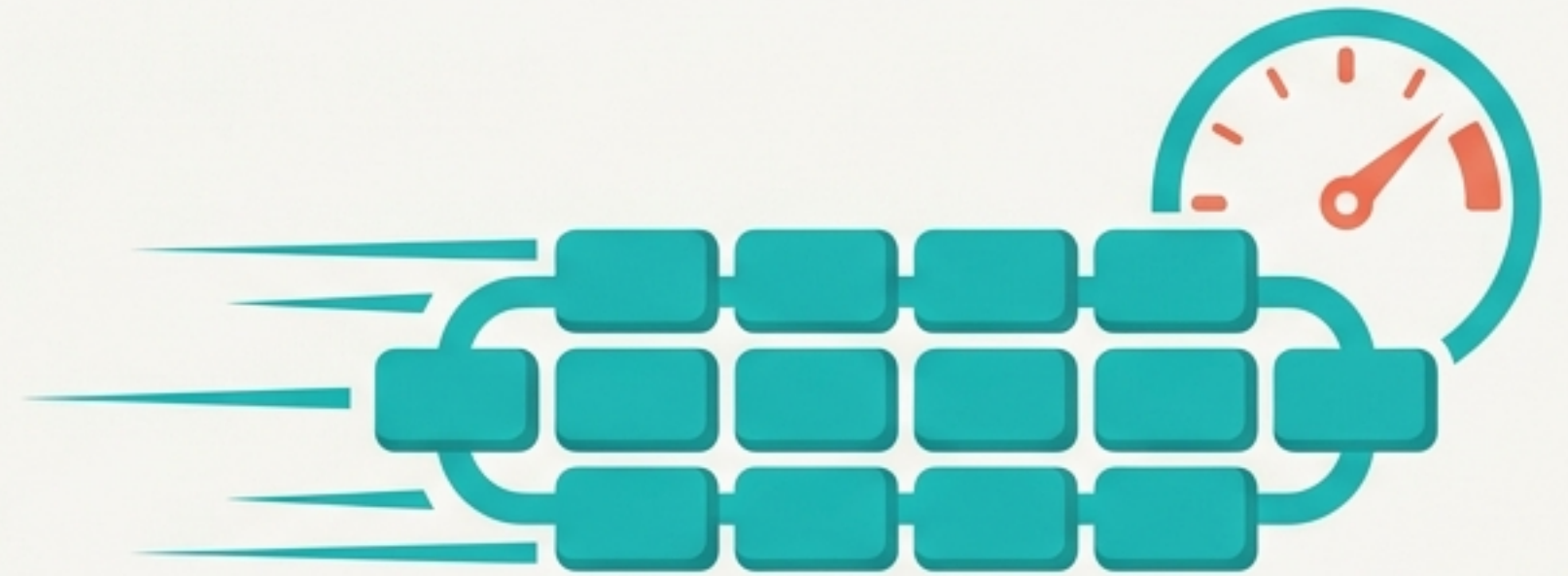
Team Leader: Mohamed Gamal Taher Saad

Members: Abdelrahman Waleed Hamdi, Ahmed Ashraf Anwar, El-Saeed Abomazbala, Abdullah Atef Abdelaziz, Hatem Doaa Ahmed Eissa

Data is heavy. Compression makes the digital world move faster.



High storage space,
heavy bandwidth usage,
slow transmission.



Saved storage space,
reduced bandwidth,
increased speed.

Without compression, the
internet would slow to a crawl:



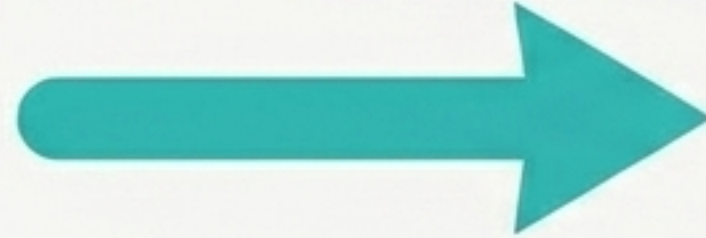
Choosing the right tool for the job: Lossless vs. Lossy

	The focus of this project	
	Lossless Compression	Lossy Compression
Is data lost?	No data is lost.	Some data is permanently removed.
Can it be restored?	Yes, original data fully restored.	No, original state is gone.
Best for...	Text, Data files, Code.	Multimedia (Audio, Video).
Examples	ZIP, PNG, Huffman Coding.	MP3, JPEG, MP4.

Huffman Coding rewards the most frequent characters

Appears frequently

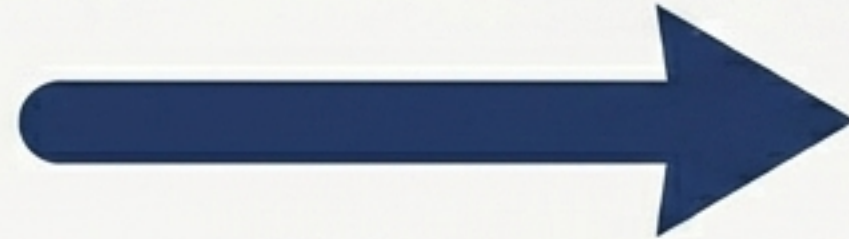
E



01

Appears rarely

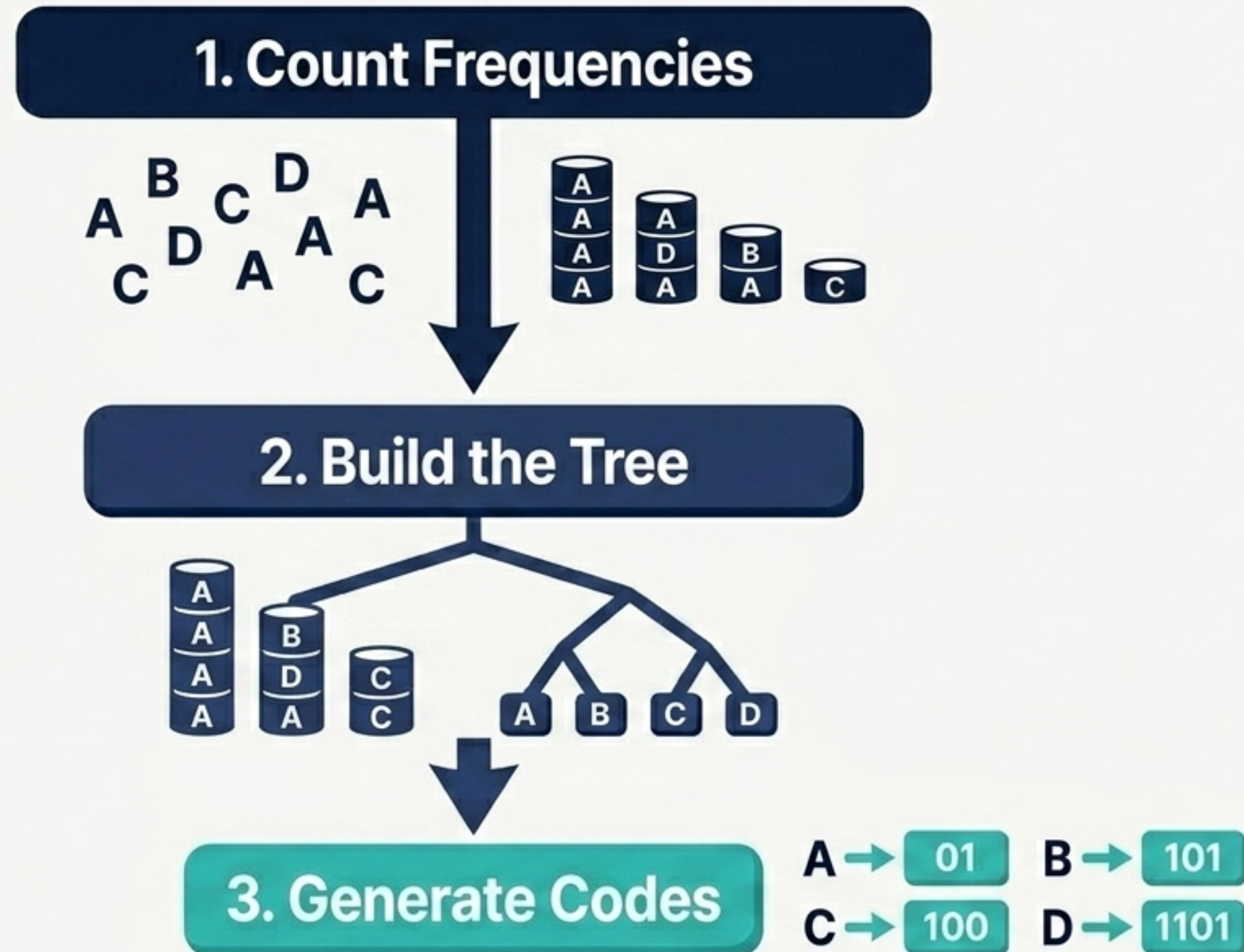
Z



110100

Key Insight Text: By giving shorter binary codes to characters that appear more frequently, the total number of bits required to store the text shrinks dramatically.

How the algorithm processes text in three simple stages

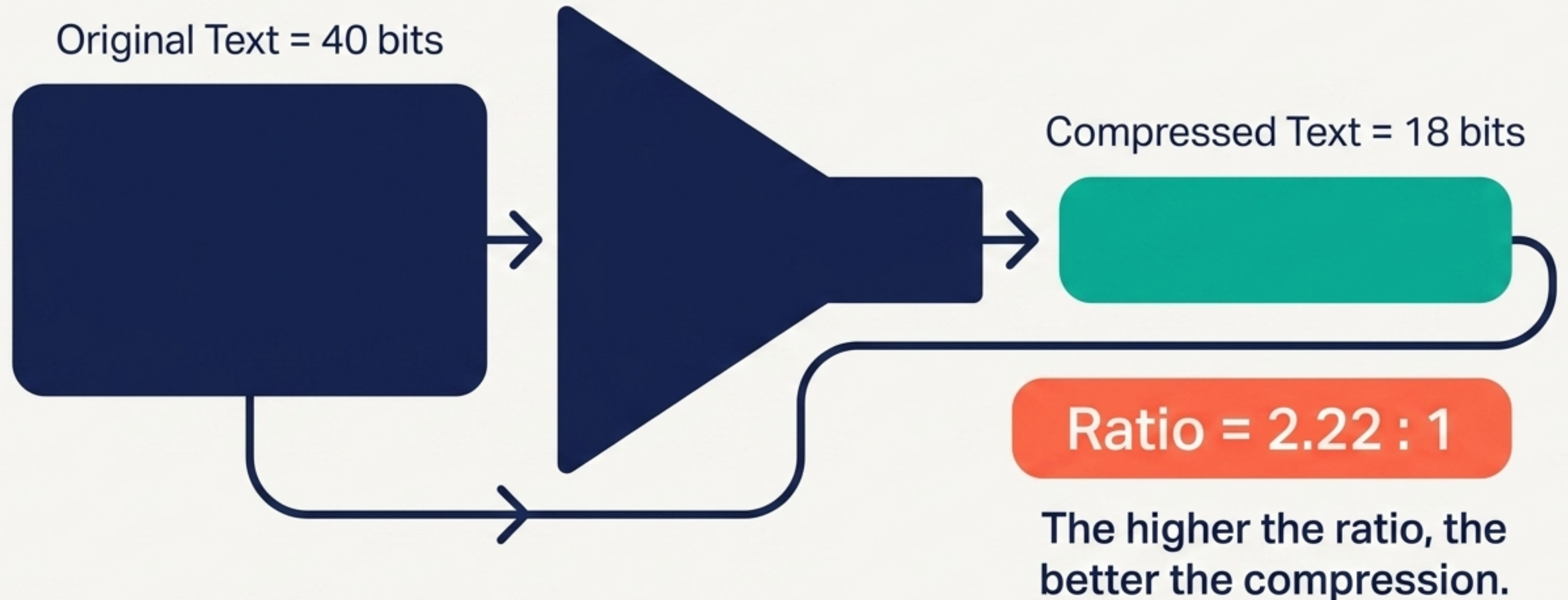


Seeing the algorithm in action with the word "hello"

Hello Example Diagram

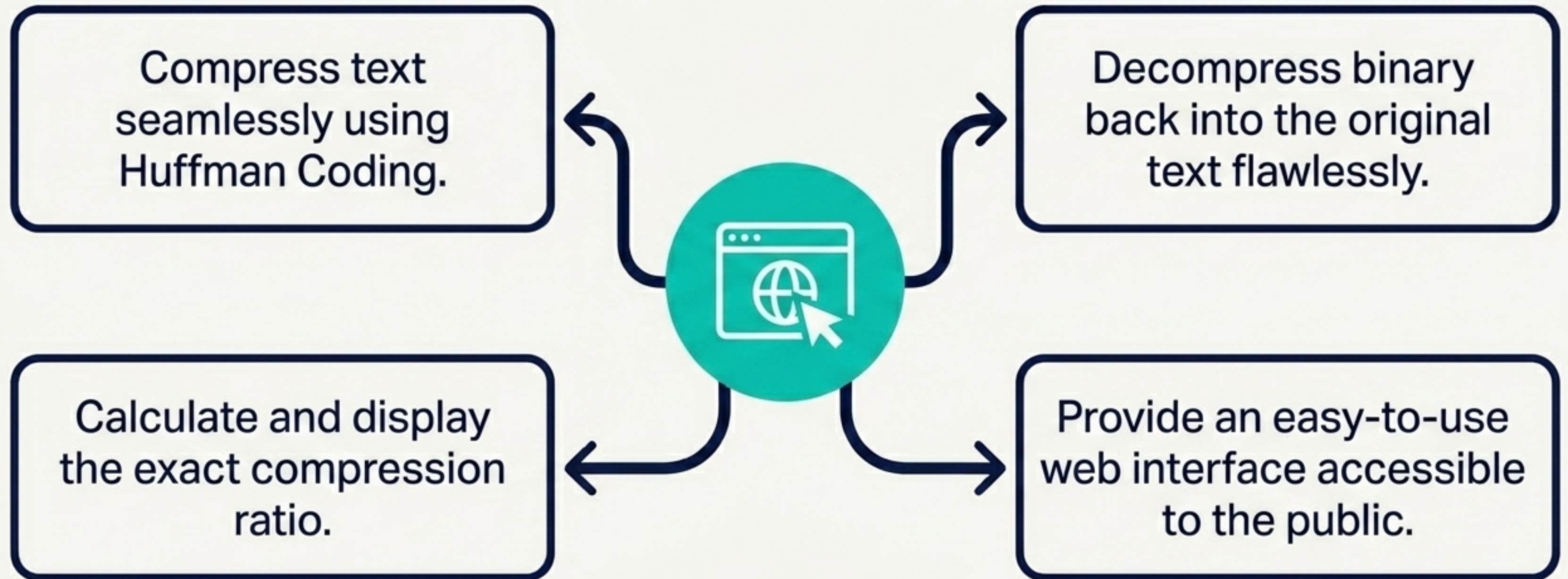


Measuring success with the Compression Ratio

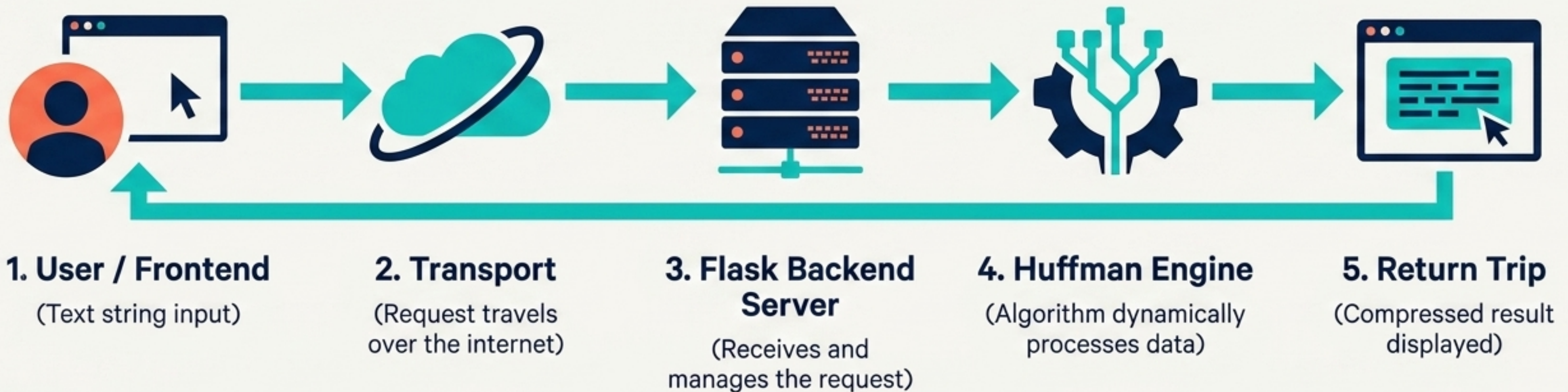


Compression Ratio = Original Size ÷ Compressed Size.

Taking the algorithm out of the textbook and onto the web



A clean, linear architecture powers the background process



The specific tools chosen to build, style, and host the platform

Frontend Interface

HTML

CSS

Clean design for dynamic user inputs.

Backend & Logic

Python

Flask

The engine room for dynamic Huffman Tree generation, encoding, and decoding.

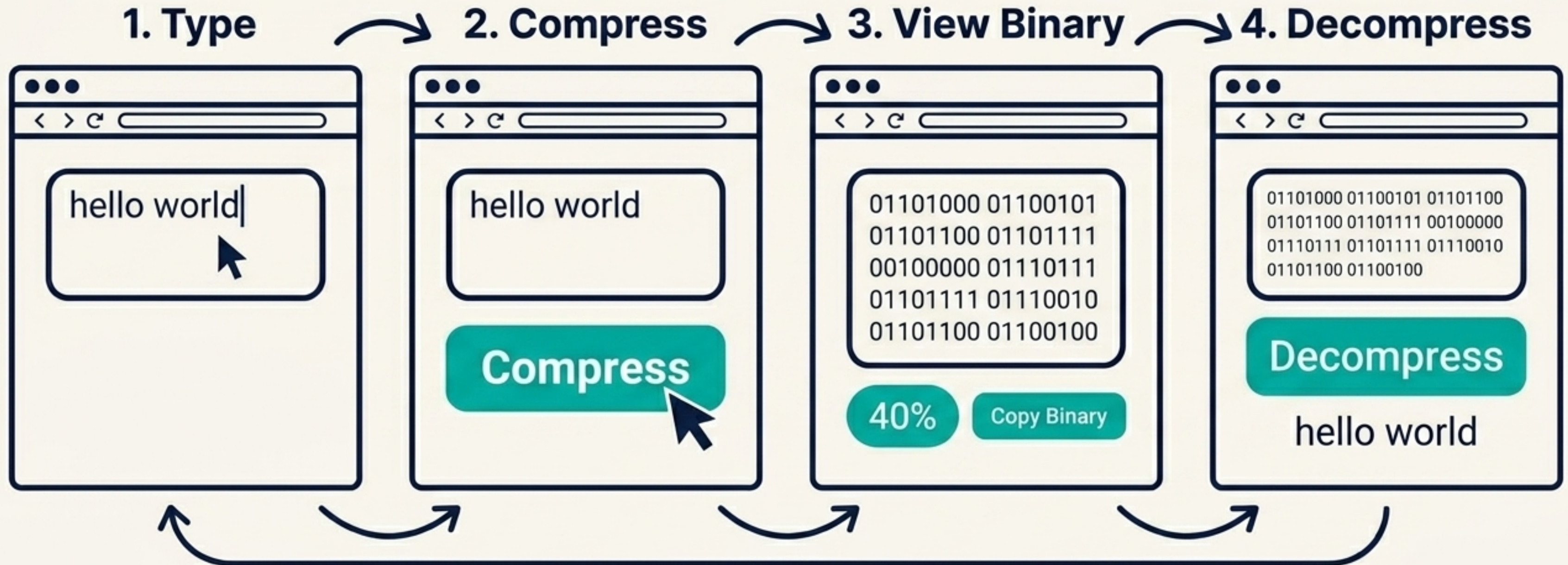
Deployment

Google Colab

LocalTunnel

Generating a public URL to allow remote, global access to a local server.

A seamless user experience from raw text to compressed binary



Navigating technical hurdles during development



State Management

Storing the dynamically generated Huffman Tree so it can be accurately referenced later for decoding



Integration

Smoothly connecting the frontend user interface with the complex backend Python logic



Environment Routing

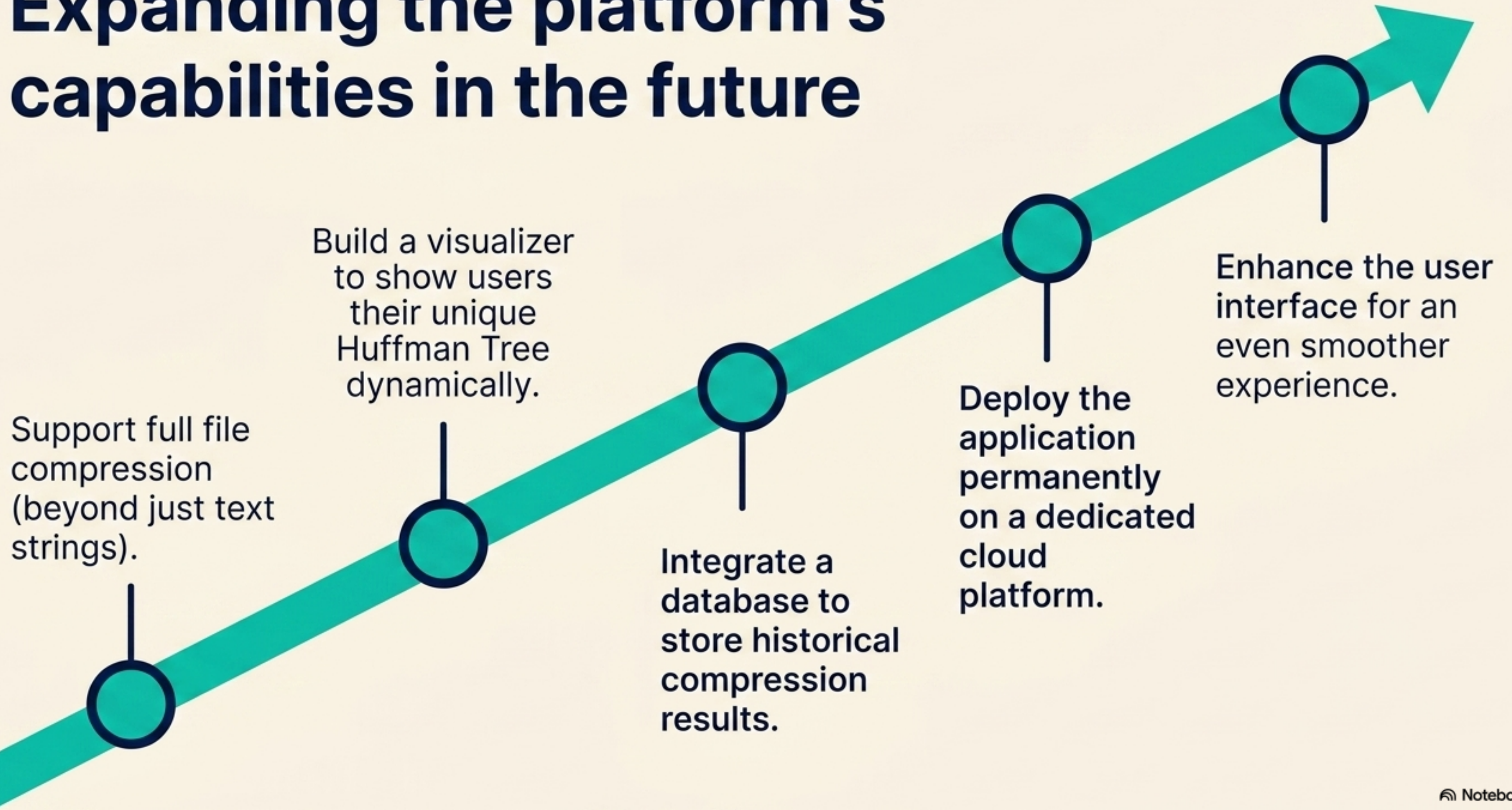
Successfully running a Flask web server from inside a Google Colab notebook



Public Access

Exposing a local server environment to the open internet for live user testing

Expanding the platform's capabilities in the future



Bridging computer science theory with accessible web engineering

